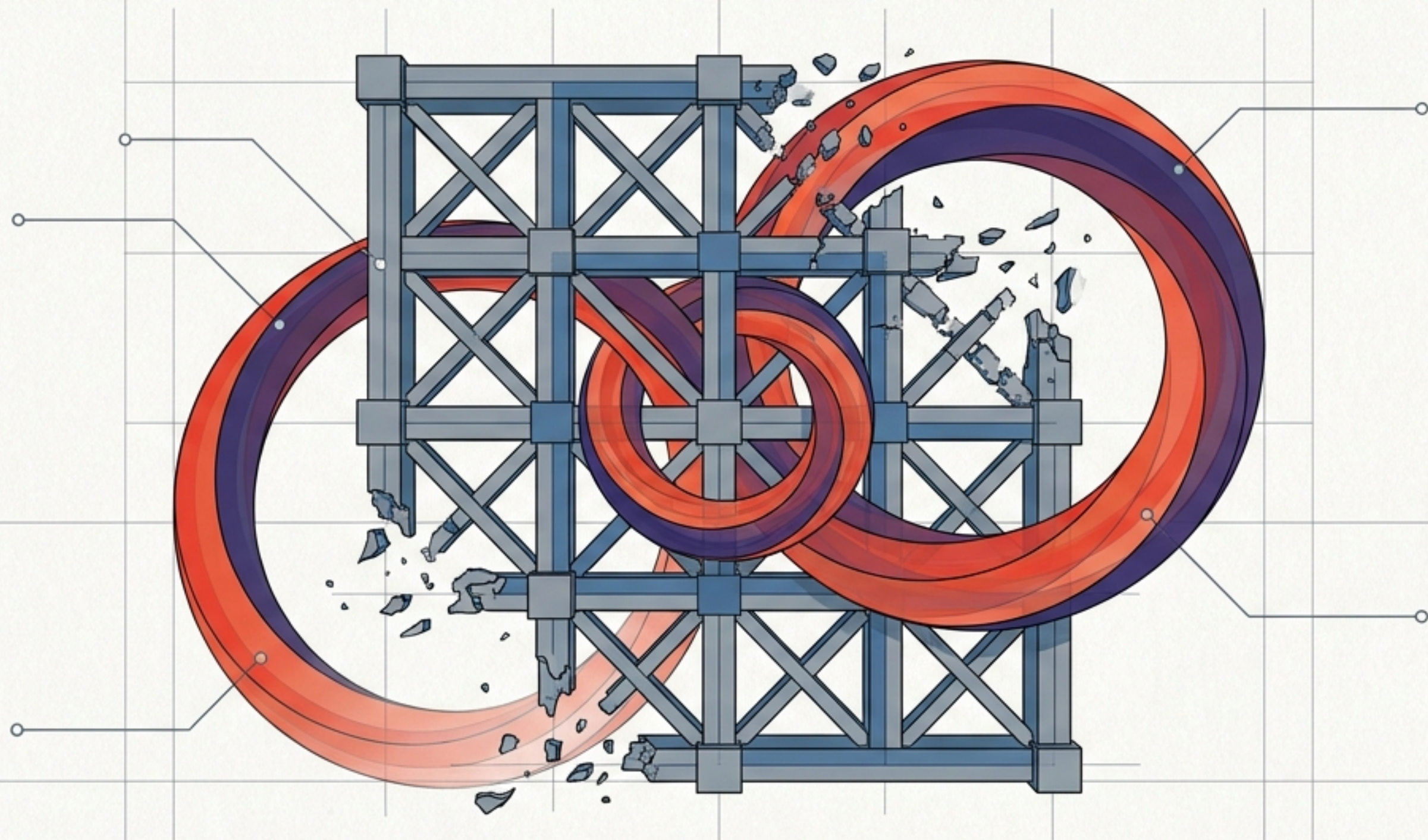
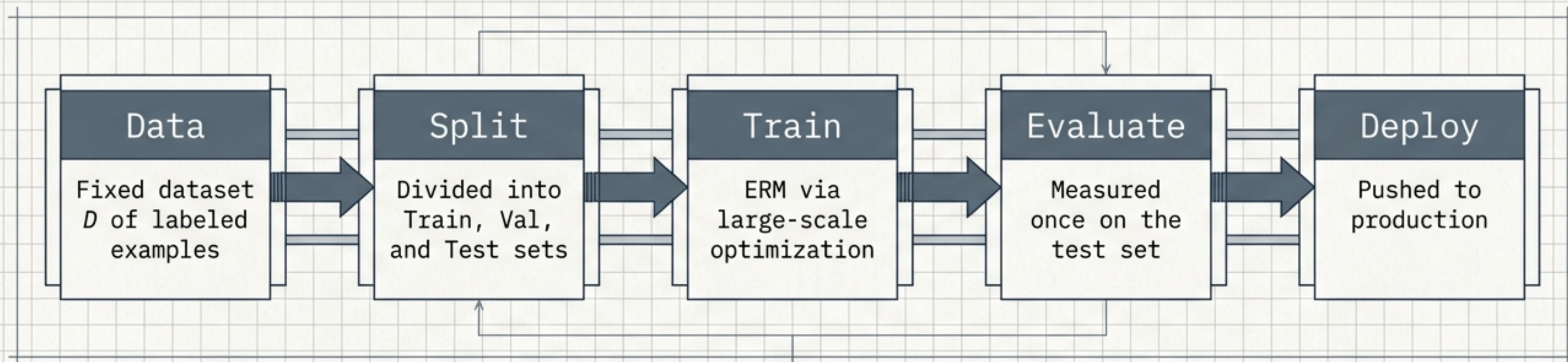




Continuous Systems

Online Learning and the Frontier of ML Scale

The Baseline: Batch Learning



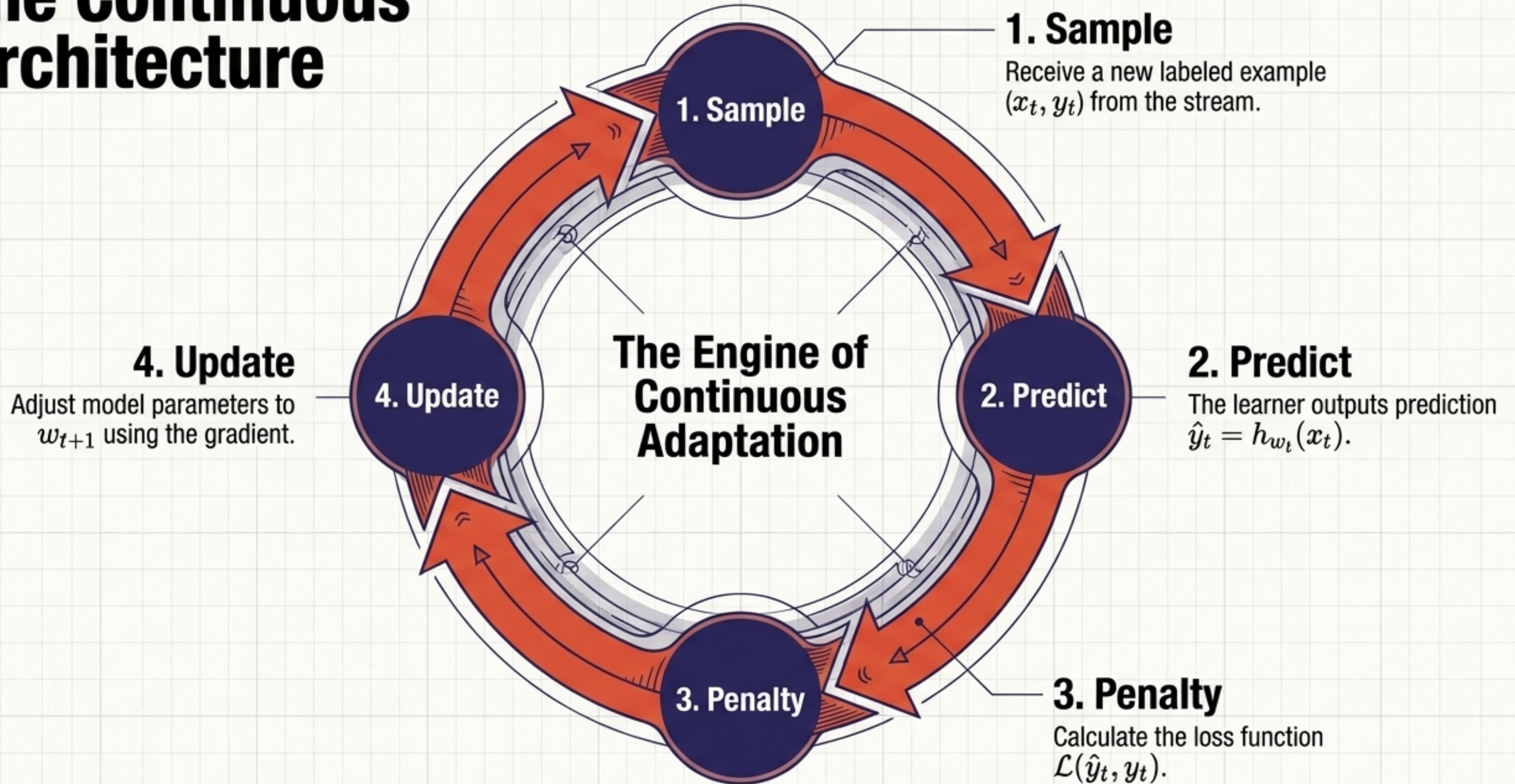
KEY LIMITATION: Once deployed, the model is **`frozen`**. It cannot adapt to new information without restarting the entire pipeline.

The Paradigm Shift

Batch Learning		Online Learning
Fixed & Finite Dataset	<i>Data Assumption</i>	Infinite Continuous Stream
Separate Training & Inference	<i>Phases</i>	Interleaved & Simultaneous
Static / Frozen in Production ❄️	<i>Model State</i>	 Continuously Evolving 
Test Error on Held-Out Data	<i>Core Metric</i>	Cumulative Regret

Online learning removes the artificial wall between training and inference.

The Continuous Architecture



* Sometimes in practice, the label y arrives a short time after the prediction.

Defining the Core Metric: Regret

The Actual Journey

The cumulative loss our evolving online learner actually incurred over time.

$$R(T) = \sum L(\hat{y}_t, y_t) - \inf_w \sum L(h_w(x_t), y_t)$$

The Theoretical Baseline

The loss a 'perfect' fixed model (ERM) would have incurred if it saw all data in advance.

Regret measures the 'extra loss' incurred by learning on the fly instead of solving the problem perfectly in a batch setting.

Online Gradient Descent (OGD)

The New State

Parameters for the next timestep.

The Step Size

How aggressively to adapt.

$$w_{t+1} = w_t - \alpha \nabla_{w_t} L(h_{w_t}(x_t), y_t)$$

The Current State

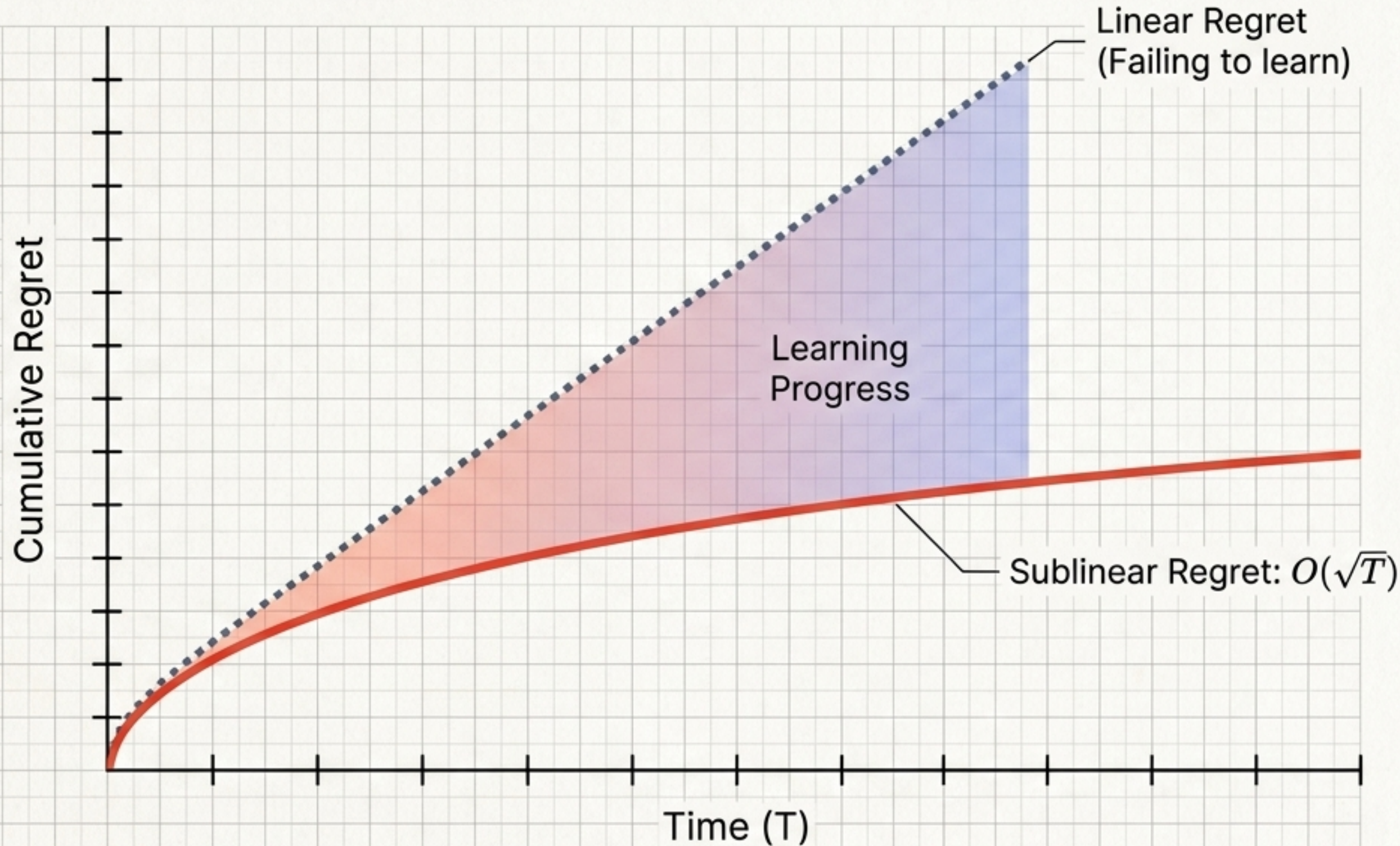
Where the model is now.

The Gradient Penalty

The direction to adjust based on the current prediction error.

Key Insight: If loss functions are convex, we can bound the distance to the optimal parameters at every step using the exact same analysis framework as SGD.

The Goal: Sublinear Regret

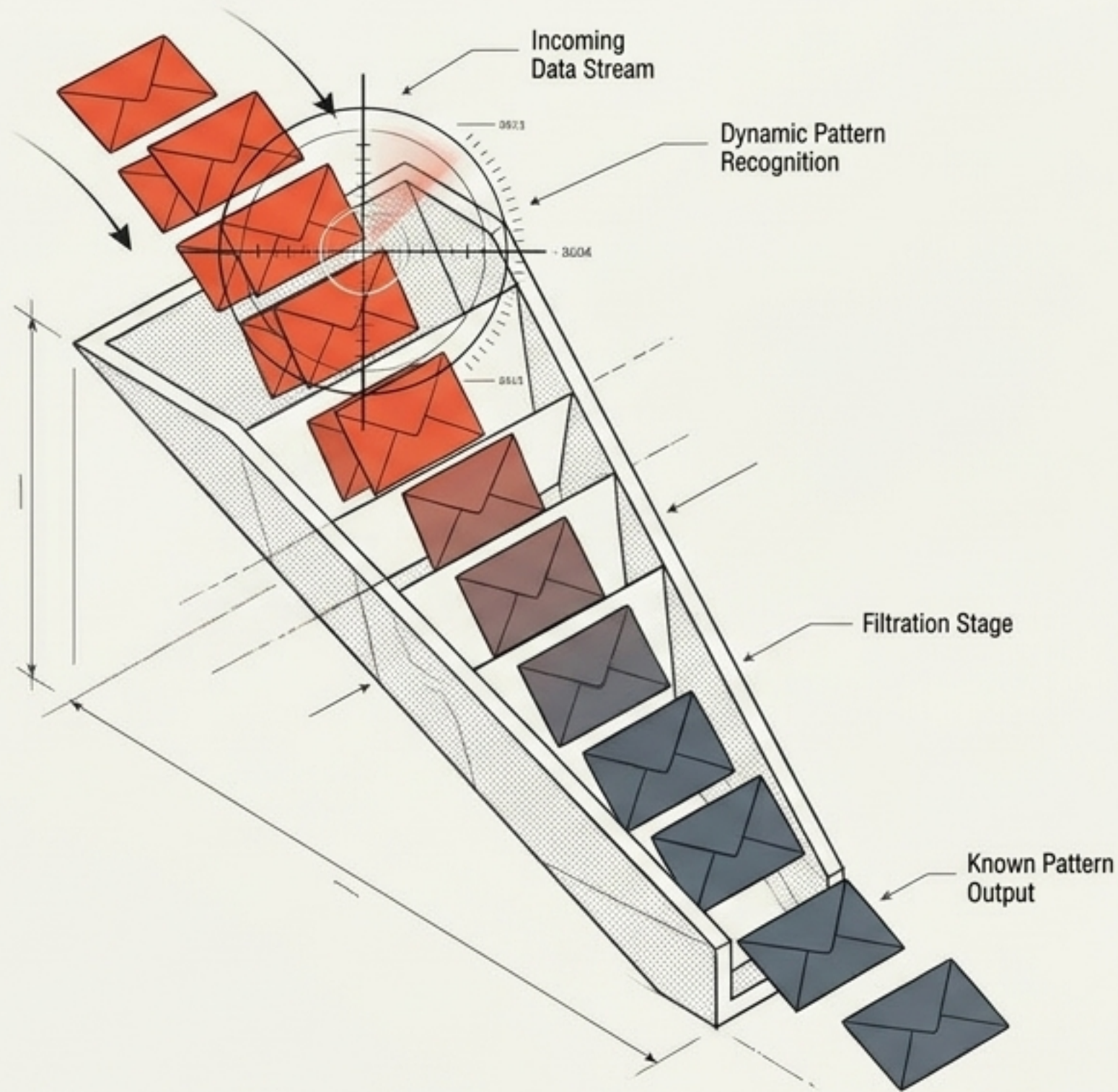


Regret inherently grows because we add new loss at every timestep.

The goal is not zero regret, but sublinear regret—meaning average regret $R(T)/T$ approaches zero.

Setting step size $\alpha = 1/\sqrt{T}$ guarantees $O(\sqrt{T})$ regret.

Applied Continuous Systems: Spam Detection



1. The Simultaneous Mandate

We must make instant predictions to filter current emails *AND* learn from them as we observe them.

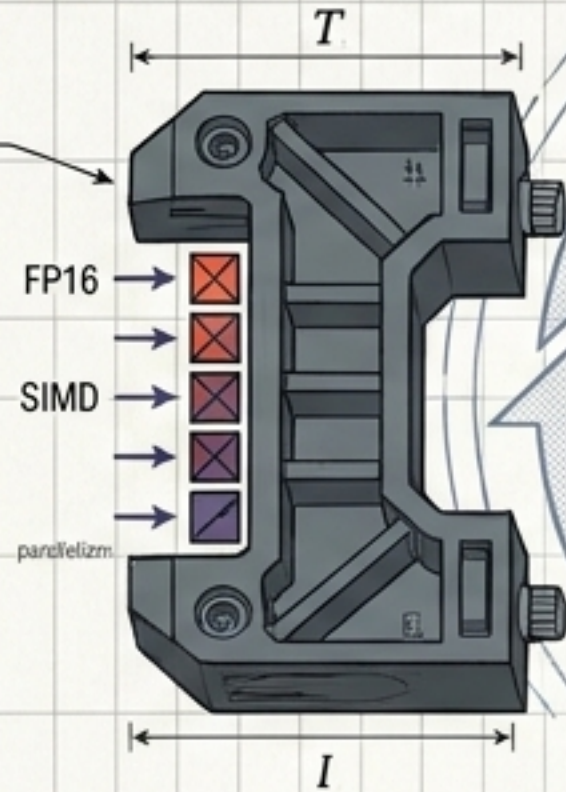
2. Distribution Shift

Online learning allows the model to “follow” small changes in the data distribution, automatically catching new classes of spam over short intervals without a full system retrain.

The Scalability Toolbelt

Hardware Acceleration

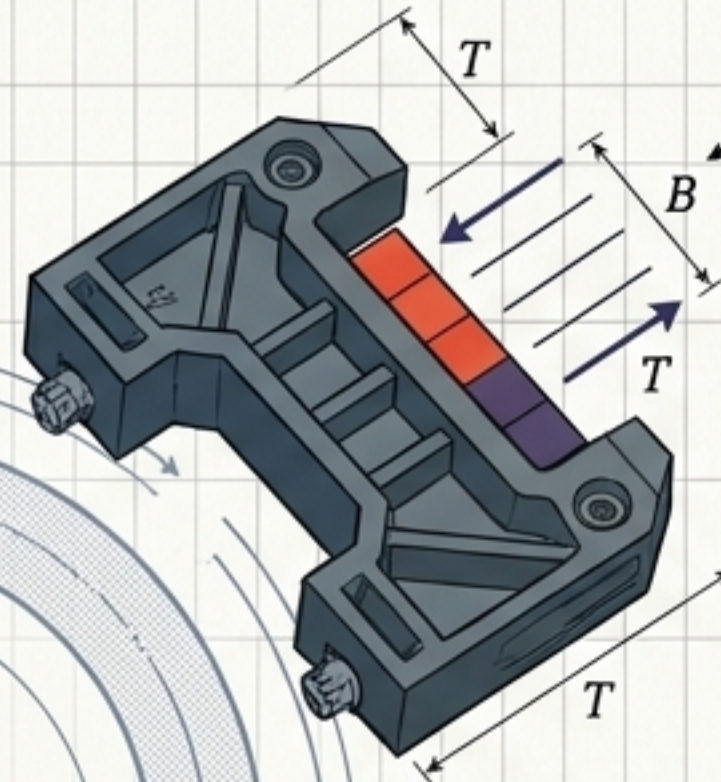
Utilizing parallelism and low-precision floating point math for real-time latency.



Online Loop

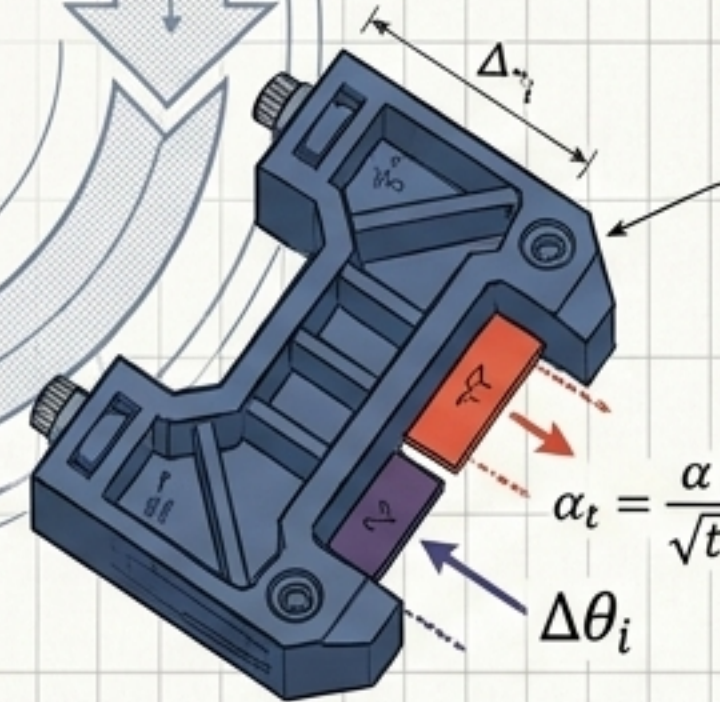
Minibatches

Processing small chunks of the stream simultaneously for efficiency.



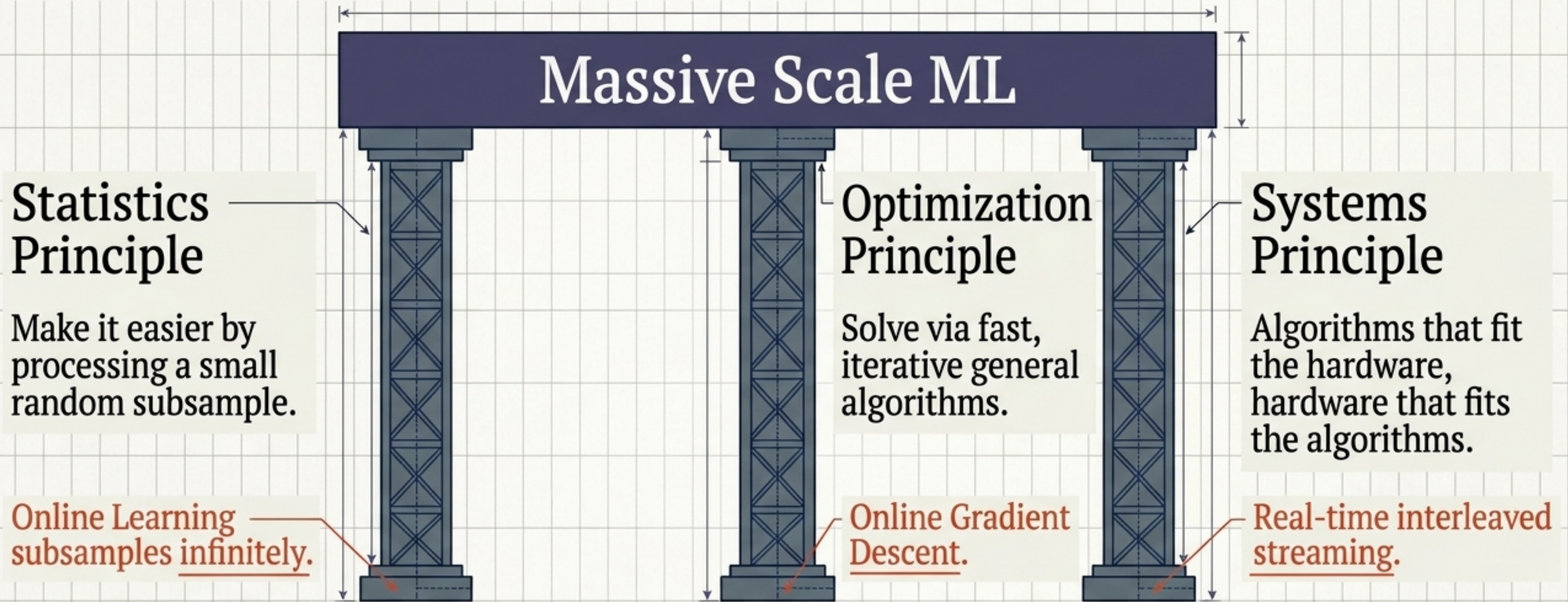
Adaptive Rates

Dynamically adjusting step sizes per parameter as the stream evolves.



The transition to online learning doesn't abandon our scalability tools; it demands we build end-to-end frameworks that support continuous ingestion.

The Temple of Scalability



Scaling machine learning is never a single discipline. It is the continuous harmony of statistics, optimization, and systems architecture.

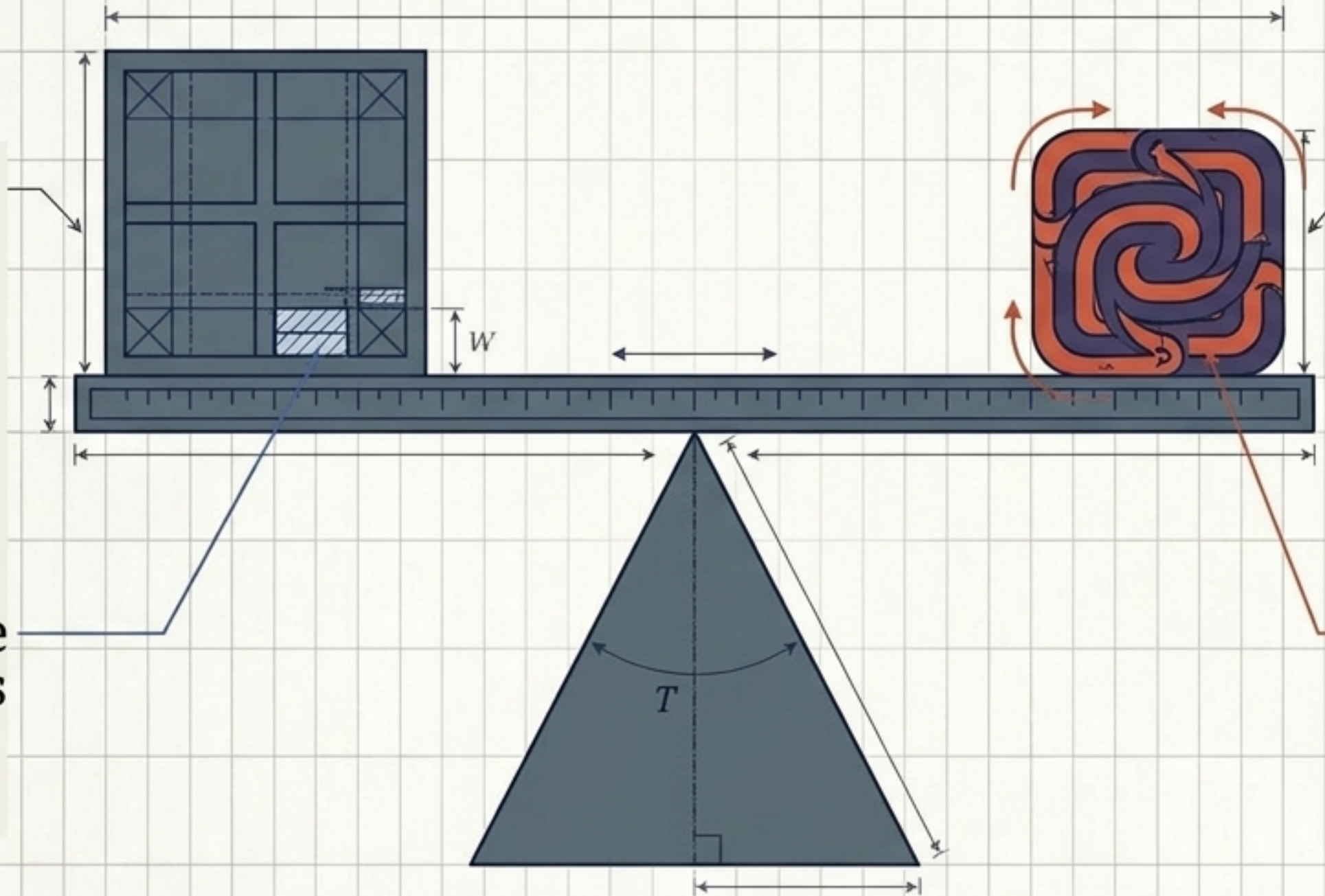
Open Problem: The Distributed Tradeoff

Heavy Systems (Large Batch Sizes)

High parallelism.

Highly efficient hardware utilization.

The Catch: Expensive communication costs requiring robust bit-compression.



Heavy Statistics (Small Batch Sizes)

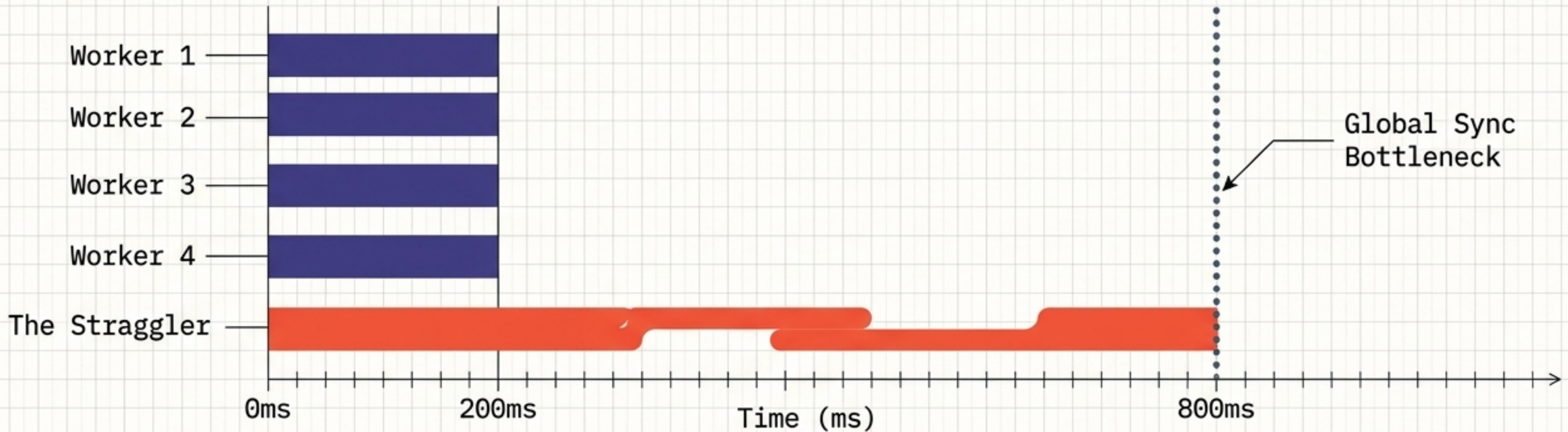
Higher gradient progress per sample.

Fundamentally better optimization.

The Catch: Underutilizes massively parallel hardware.

To scale distributed ML, we must bridge the gap between statistical progress and systems parallelism.

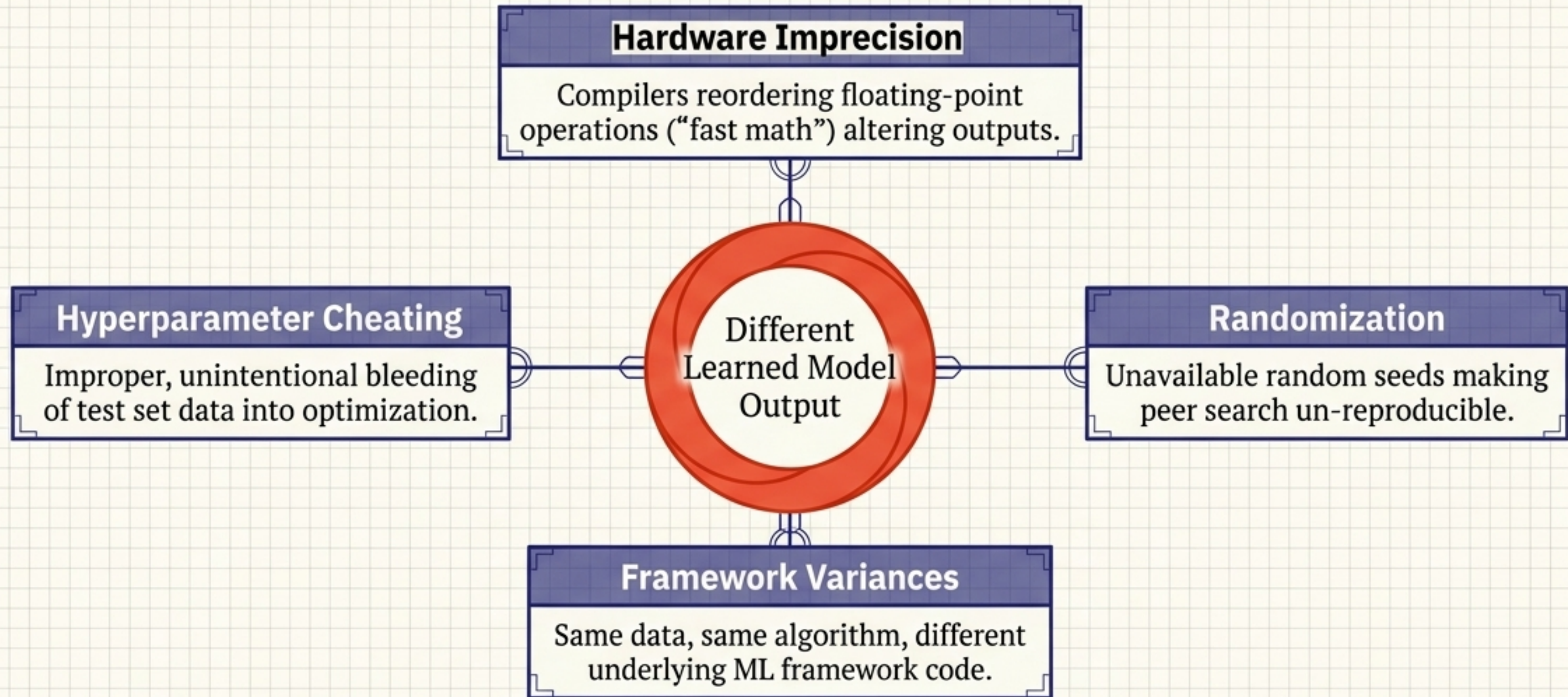
Open Problem: The Straggler Bottleneck



Datacenters are heterogeneous. When running many workers in parallel, global performance degrades to the speed of the slowest machine.

The Open Question: How do we distribute workloads across mismatched workers while retaining mathematical performance guarantees?

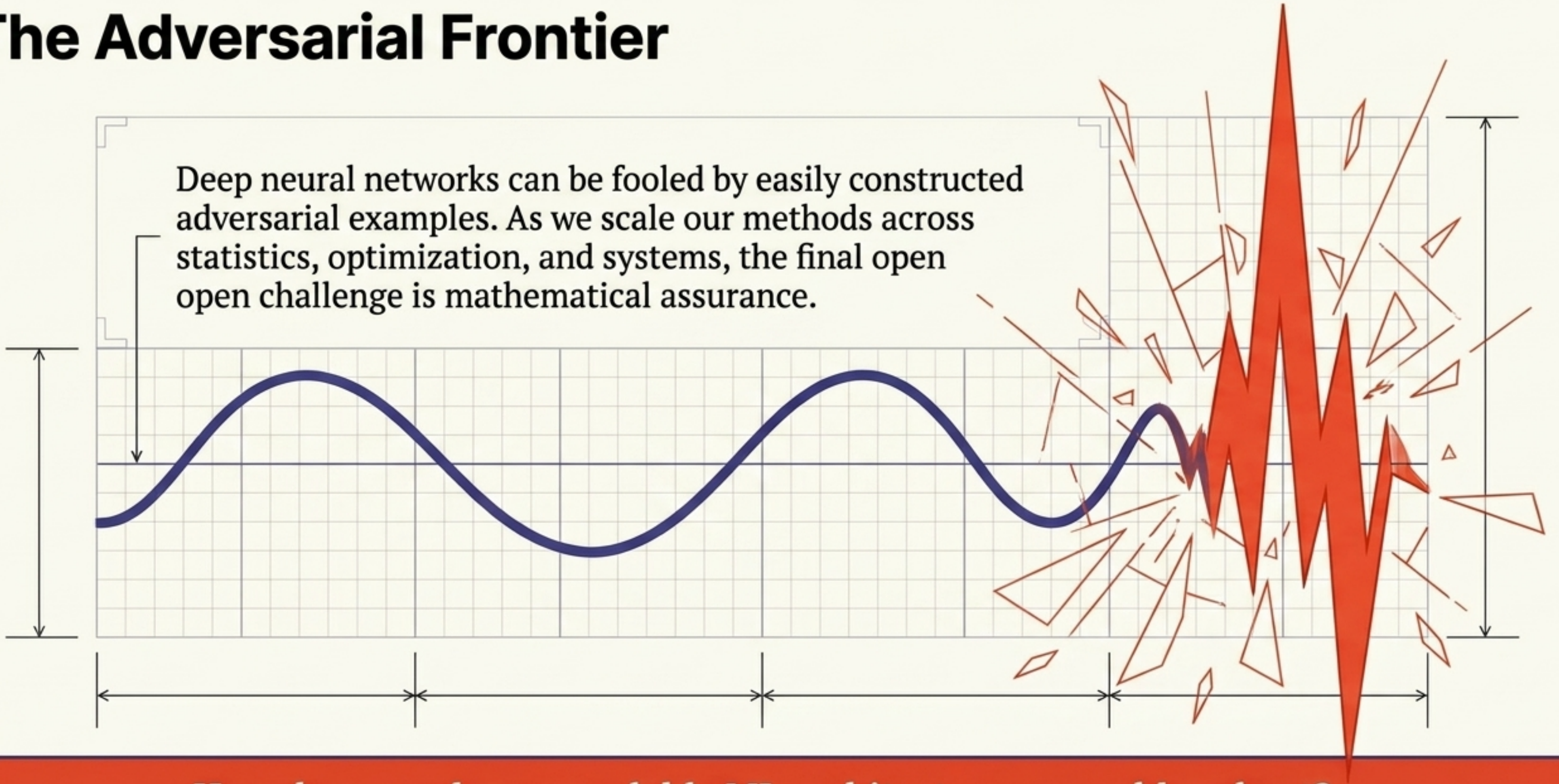
Open Problem: The Reproducibility Web



ML models rarely crash; they learn around bugs, making silent, untraceable errors the hardest problem in systems engineering.

The Adversarial Frontier

Deep neural networks can be fooled by easily constructed adversarial examples. As we scale our methods across statistics, optimization, and systems, the final open challenge is mathematical assurance.



How do we make our scalable ML architectures provably robust?
The systems of tomorrow must be as resilient as they are massive.